



Tecnológico de Monterrey

**Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla**

Materia: Fundamentación de robótica

Clave: TE3001B

Grupo: 101

Profesores:

Alfredo García Suárez
Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios

Challenge 2

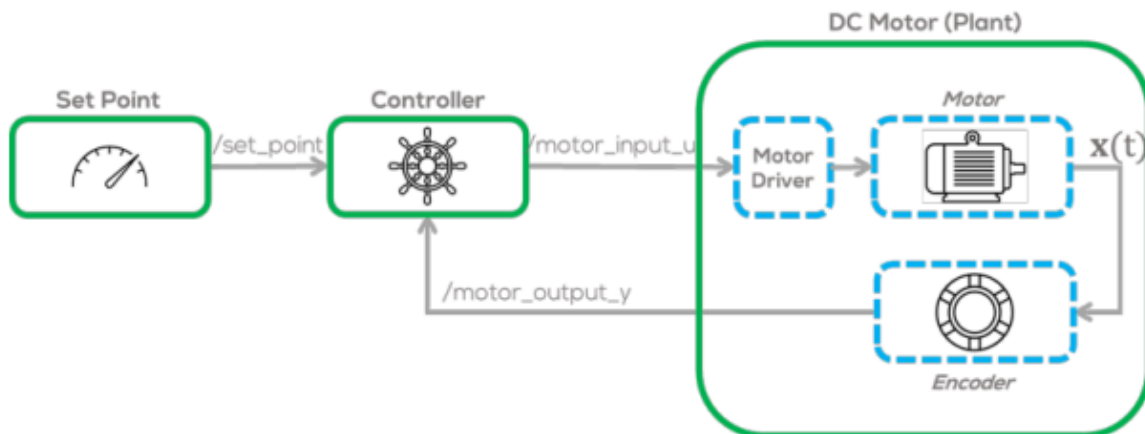
Alumnos

Omar Perez Dominguez | A01738306
Ezequiel Luna Trejo | A01738153
Antonio Méndez Rodríguez | A01738269

22 de Febrero De 2026

Resumen

Este proyecto consistió en el diseño y puesta en marcha de un sistema de control para regular la velocidad de un motor de CD utilizando ROS 2 Humble. A partir de una base de paquetes proporcionada, se desarrolló un nodo controlador (`controller.py`) programado en Python con la librería NumPy y un archivo de automatización (`motor_launch.py`). El objetivo central fue cerrar el lazo de control para que el motor reaccione automáticamente a los cambios en la señal de referencia. Mediante el uso de *rqt_plot* y *rqt_graph*, podemos graficar nuestras señales con el propósito de comprobar que el sistema es capaz de seguir la trayectoria deseada de forma estable, validando así nuestro sistema de control y la correcta integración de los nodos y el intercambio de mensajes personalizados.

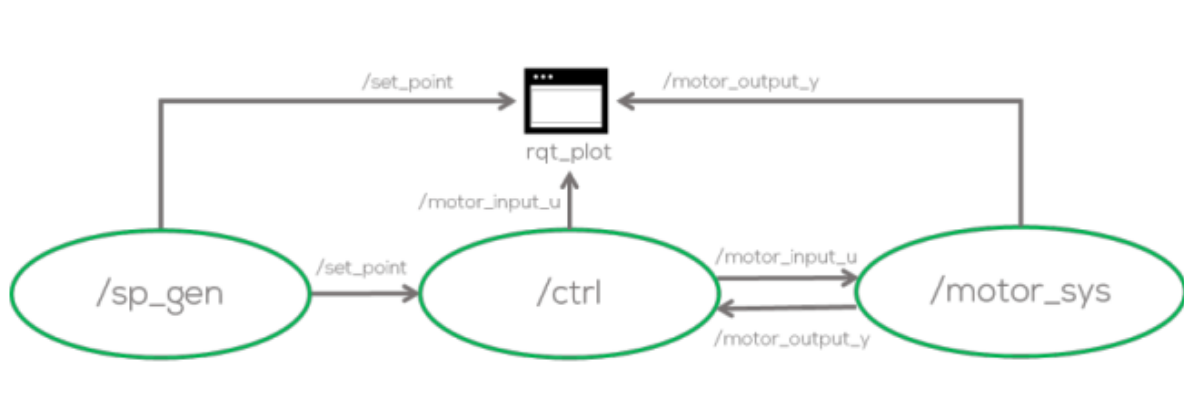


Objetivo General

Diseñar y validar un sistema de control de velocidad en lazo cerrado para un motor de corriente directa mediante el framework ROS 2 Humble, asegurando la correcta implementación del sistema de control que siga la señal de manera precisa y estable a través de la programación de un nodo controlador personalizado en Python.

Objetivos específicos

- Programación Robusta: Desarrollar un nodo de control (***controller.py***) que utilice la librería NumPy para el cálculo de un algoritmo PI
- Gestión Dinámica: Implementar el uso de parámetros de ROS 2 para permitir la sintonización de las ganancias K_p y K_i en tiempo de ejecución, optimizando el desempeño del motor sin interrumpir el proceso
- Automatización del Sistema: Crear un archivo de lanzamiento (***motor_launch.py***) que integre y configure simultáneamente los nodos de la planta, el generador de referencias y el controlador, estableciendo las condiciones iniciales del sistema.
- Análisis y Monitoreo: Utilizar herramientas de **rqt_plot** y **rqt_graph** para graficar las señales de control y retroalimentación, para poder verificar que el error en estado estacionario sea el menor posible y que la topología de la red de nodos sea correcta.



Introducción: Procesamiento de Señales en ROS 2

En el ámbito de la robótica y la automatización, el intercambio eficiente de datos y la precisión en la respuesta mecánica son pilares fundamentales. Este proyecto se desarrolla bajo el ecosistema de ROS 2 (Robot Operating System 2), un middleware que facilita la comunicación entre procesos mediante una arquitectura de nodos. El objetivo es implementar un sistema de control de velocidad para un motor de corriente directa, integrando conceptos de redes de comunicación y teoría de control.

Conceptos de Arquitectura en ROS 2

- **Namespaces:** Son herramientas de organización que permiten agrupar nodos, tópicos y servicios bajo un nombre común. Esto evita colisiones de nombres en proyectos complejos y permite que múltiples instancias de un mismo sistema (por ejemplo, varios motores) coexistan de forma independiente.
- **Parameters:** Son valores de configuración que los nodos utilizan en tiempo de ejecución. En este trabajo, se implementan para definir las ganancias K_p y K_i , permitiendo al usuario modificar el comportamiento del controlador dinámicamente sin necesidad de recompilar el código.
- **Custom Messages:** Representan estructuras de datos personalizadas para la transmisión de información. En este caso, se utilizan mensajes de tipo *std_msgs/Float32* y otros definidos en el paquete *custom_interfaces* para comunicar la velocidad del motor, la referencia y la señal de voltaje.

Control de Lazo Cerrado y el Controlador PI

Una parte muy importante de la investigación es el Controlador PI (Proporcional-Integral). Este sistema de control funciona bajo un esquema de lazo cerrado, donde el nodo controlador recibe continuamente la velocidad real del motor y la compara con la velocidad deseada.

- **Acción Proporcional (P):** Genera una respuesta inmediata basada en el error actual; a mayor error, mayor esfuerzo de control.
- **Acción Integral (I):** Se encarga de acumular el error a lo largo del tiempo. Su función principal es eliminar el error en estado estacionario, asegurando que el sistema trate de alcanzar exactamente la meta establecida.

Sintonización del Controlador

La sintonización es el proceso de ajustar las ganancias K_p y K_i , para obtener una respuesta óptima. En este reporte, la sintonía se realizó de manera empírica, observando la respuesta del sistema en *rqt_plot*. Se buscó un equilibrio donde el motor alcanzará la velocidad deseada rápidamente (dominio de la parte proporcional) pero sin oscilaciones excesivas, permitiendo que la parte integral estabiliza el valor final de forma precisa.

Solución del problema

Análisis de la Planta y Configuración de Parámetros En lugar de un modelado gráfico, la solución partió del análisis de las especificaciones técnicas proporcionadas por Manchester Robotics para el motor. Se identificó que el sistema responde a una planta de primer orden, lo cual se tradujo directamente en la configuración del nodo *dc_motor* dentro del archivo de lanzamiento. Se establecieron parámetros críticos como la ganancia del sistema ($K = 2.16$) y la constante de tiempo ($\tau = 0.05$), los cuales dictan la rapidez y la magnitud de la respuesta del motor ante un estímulo de voltaje.

Desarrollo del Nodo Controlador (*controller.py*)

La implementación del nodo controlador se realizó en Python mediante una arquitectura orientada a objetos en donde buscamos la eficiencia matemática y la flexibilidad, utilizando un algoritmo de control PI (Proporcional-Integral) diseñado para equilibrar una respuesta dinámica rápida con la eliminación total del error en estado estacionario. El sistema ejecuta un lazo de control de alta frecuencia (100 Hz) mediante un *timer* de ROS 2, donde se emplea la librería NumPy para procesar la señal de error y aplicar una técnica de Anti-windup a través de la función *np.clip*, la cual limita la acumulación integral entre -100.0 y 100.0 para prevenir saturaciones e inestabilidad en el actuador. Asimismo, se integró una interfaz de parámetros dinámicos que permite la sintonización al momento de las ganancias K_p y K_i , respaldada por un *parameters_callback* que valida en tiempo real que los valores sean positivos, garantizando así una operación segura y un seguimiento de trayectoria estable según lo verificado en las pruebas experimentales, anexo una imagen del código elaborado sobre el nodo controller.py.

```

1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import Float32
4 from rcl_interfaces.msg import SetParametersResult
5 import numpy as np
6
7
8 class Controller(Node):
9
10     def __init__(self):
11         super().__init__('ctrl')
12
13         self.declare_parameter('Kp', 0.5)
14         self.declare_parameter('Ki', 0.1)
15
16         # Get Parameters
17         self.Kp = self.get_parameter('Kp').value
18         self.Ki = self.get_parameter('Ki').value
19
20
21         self.y = 0.0
22         self.sp = 0.0
23         self.error_sum = 0.0
24
25         # Publisher
26         self.pub_u = self.create_publisher(Float32, 'motor_input_u', 10)
27
28         # Subscribers
29
30         self.sub_y = self.create_subscription(
31             Float32,
32             'motor_speed_y',
33             self.callback_y,
34             10
35         )
36
37         self.sub_sp = self.create_subscription(
38             Float32,
39             'set_point',
40             self.callback_sp,
41             10
42         )
43
44         # Timer (loop de control)
45         self.sample_time = 0.01 # igual que motor_sys
46         self.timer = self.create_timer(self.sample_time, self.control_loop)
47
48         # Parameter Callback
49         self.add_on_set_parameters_callback(self.parameters_callback)
50
51         self.get_logger().info('Controller Node Started ')
52
53

```

```

57
58     def callback_sp(self, msg):
59         self.sp = msg.data
60
61         # Control Loop
62     def control_loop(self):
63
64         # Error
65         error = self.sp - self.y
66
67         # Integral (acumulación)
68         self.error_sum += error
69
70         # Anti-windup
71         self.error_sum = np.clip(self.error_sum, -100.0, 100.0)
72
73         # Control PI
74         u = self.Kp * error + self.Ki * self.error_sum
75
76         # Publicar señal de control
77         msg = Float32()
78         msg.data = float(u)
79         self.pub_u.publish(msg)
80
81         # Parameter validation
82     def parameters_callback(self, params):
83         for param in params:
84
85             if param.name == "Kp":
86                 if param.value < 0.0:
87                     self.get_logger().warn("Invalid Kp! Cannot be negative.")
88                     return SetParametersResult(successful=False, reason="Kp < 0")
89                 else:
90                     self.Kp = param.value
91                     self.get_logger().info(f"Kp updated to {self.Kp}")
92
93             if param.name == "Ki":
94                 if param.value < 0.0:
95                     self.get_logger().warn("Invalid Ki! Cannot be negative.")
96                     return SetParametersResult(successful=False, reason="Ki < 0")
97                 else:
98                     self.Ki = param.value
99                     self.get_logger().info(f"Ki updated to {self.Ki}")
100         return SetParametersResult(successful=True)
101
102 # Main
103 def main(args=None):
104     rclpy.init(args=args)
105
106     node = Controller()
107
108     try:
109         rclpy.spin(node)
110     except KeyboardInterrupt:
111         pass
112     finally:
113         node.destroy_node()
114         rclpy.shutdown()
115
116 if __name__ == '__main__':
117     main()

```

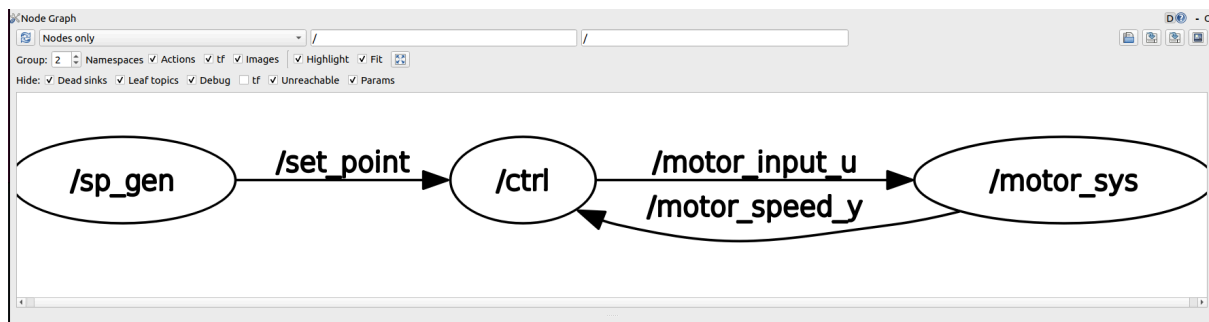
Automatización (*motor_launch.py*)

La automatización y orquestación del sistema se consolidó mediante un script de lanzamiento (*motor_launch.py*), el cual permite la gestión centralizada de los ciclos de vida de los nodos y la inyección de configuraciones críticas para la simulación. La arquitectura del archivo destaca por la instanciación del nodo *motor_sys*, donde se definieron de forma explícita los parámetros físicos de la planta, incluyendo una ganancia del sistema de 2.16 y una constante de tiempo de 0.05 s, asegurando que el modelo del motor de CD opere bajo condiciones dinámicas realistas y consistentes en cada ejecución. Asimismo, el script coordina la ejecución concurrente del generador de referencias (*sp_gen*) y el nodo controlador personalizado (*ctrl*), utilizando las propiedades *emulate_tty* y *output='screen'* para garantizar que el flujo de datos y los mensajes de diagnóstico sean legibles directamente en la consola. Esta metodología de despliegue no solo optimiza el flujo de trabajo al eliminar la necesidad de ejecución manual por terminales separadas, sino que también garantiza la integridad de la red de comunicación al sincronizar el arranque de la planta con la activación del lazo de control PI, facilitando así la captura síncrona de datos para su posterior análisis en herramientas de visualización.

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4 def generate_launch_description():
5
6     motor_node = Node(
7         name="motor_sys",
8         package='motor_control',
9         executable='dc_motor',
10        emulate_tty=True,
11        output='screen',
12        parameters=[{
13            'sample_time': 0.01,
14            'sys_gain_K': 2.16,
15            'sys_tau_T': 0.05,
16            'initial_conditions': 0.0,
17        }]
18    )
19
20    sp_node = Node(
21        name="sp_gen",
22        package='motor_control',
23        executable='set_point',
24        emulate_tty=True,
25        output='screen',
26    )
27
28    ctrl_node = Node(
29        name="ctrl",
30        package='motor_control',
31        executable='controller',
32        emulate_tty=True,
33        output='screen',
34    )
35
36    l_d = LaunchDescription([
37        motor_node,
38        sp_node,
39        ctrl_node
40    ])
41
42    return l_d
```

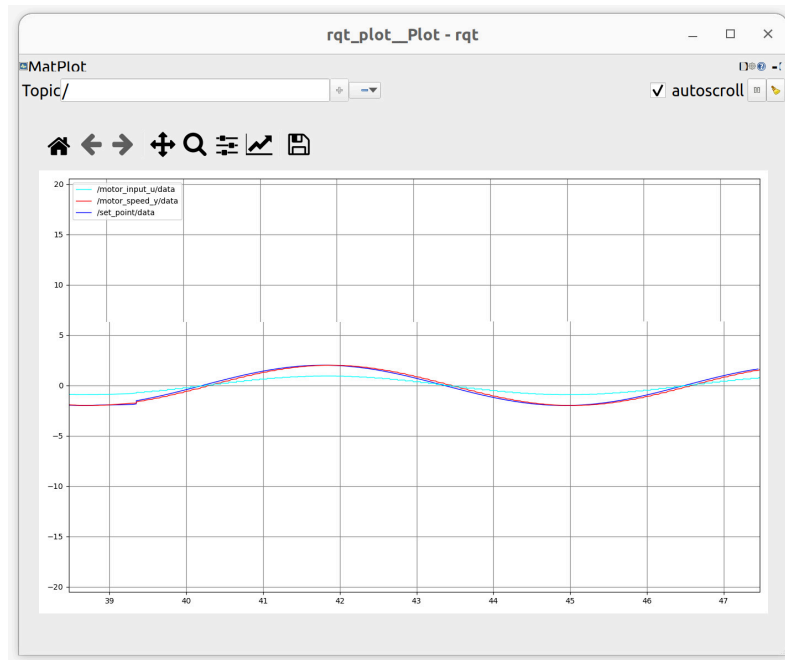
Resultados

La validación de la arquitectura de comunicación se realizó mediante la generación del diagrama de computación del sistema a través de la herramienta *rqt_graph*, la cual permite visualizar de manera clara el flujo de datos y la interconectividad entre los procesos de ROS 2. En el diagrama obtenido, se identifica un lazo de control cerrado perfectamente estructurado donde el nodo de referencia, denominado *n_sp_gen*, actúa como el generador de trayectoria al publicar la velocidad deseada en el tópico */set_point*. Esta señal es recibida por el nodo controlador central, */ctrl*, el cual desempeña un rol crítico de intermediario al estar suscrito simultáneamente a la referencia y a la retroalimentación proveniente del motor a través del tópico */motor_speed_y*. Como resultado del procesamiento interno mediante el algoritmo PI y la lógica de Anti-windup implementada en Python, el controlador publica la señal de mando calculada en el tópico */motor_input_u*, la cual es consumida por el nodo de planta *n_motor_sys*. Este último cierra el ciclo de retroalimentación al enviar la velocidad real alcanzada de vuelta al controlador, confirmando así que la topología de la red es robusta, que no existen nodos aislados y que el intercambio de mensajes personalizados se ejecuta de forma síncrona y eficiente bajo el esquema de diseño establecido.



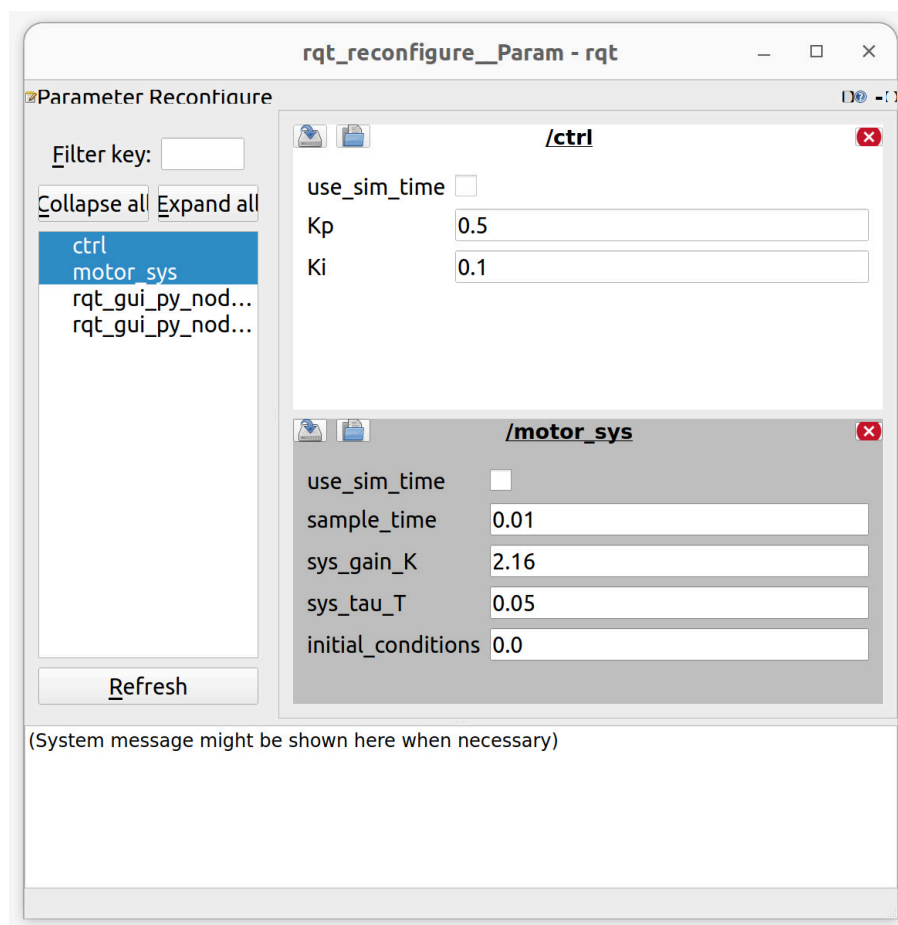
El análisis de la respuesta temporal del sistema evidencia un desempeño altamente satisfactorio del controlador PI en el seguimiento de trayectorias dinámicas. Se observa que la velocidad real del motor (representada por la señal roja) converge de manera casi instantánea con la señal de referencia (línea azul), manteniendo un error de seguimiento despreciable a lo largo de todo el ciclo de operación. Esta precisión es una prueba directa de la efectividad de la acción integral programada, la cual compensa las perturbaciones y la dinámica del modelo físico para eliminar el error en estado estacionario que caracterizaría a un control puramente proporcional. Asimismo, la señal de control o voltaje de entrada (línea cian) presenta una evolución suave y coherente con las variaciones de la referencia, lo que indica que la sintonización de las ganancias K_p y K_i es óptima para la constante de tiempo de la planta.

La ausencia de oscilaciones divergentes o sobretiros pronunciados en las crestas de la señal confirma no solo la estabilidad del lazo cerrado, sino también la correcta implementación de la lógica de Anti-windup, la cual previene que la saturación del actuador degrade la respuesta del sistema ante cambios continuos en la consigna, validando así la robustez del diseño desarrollado para el reto.

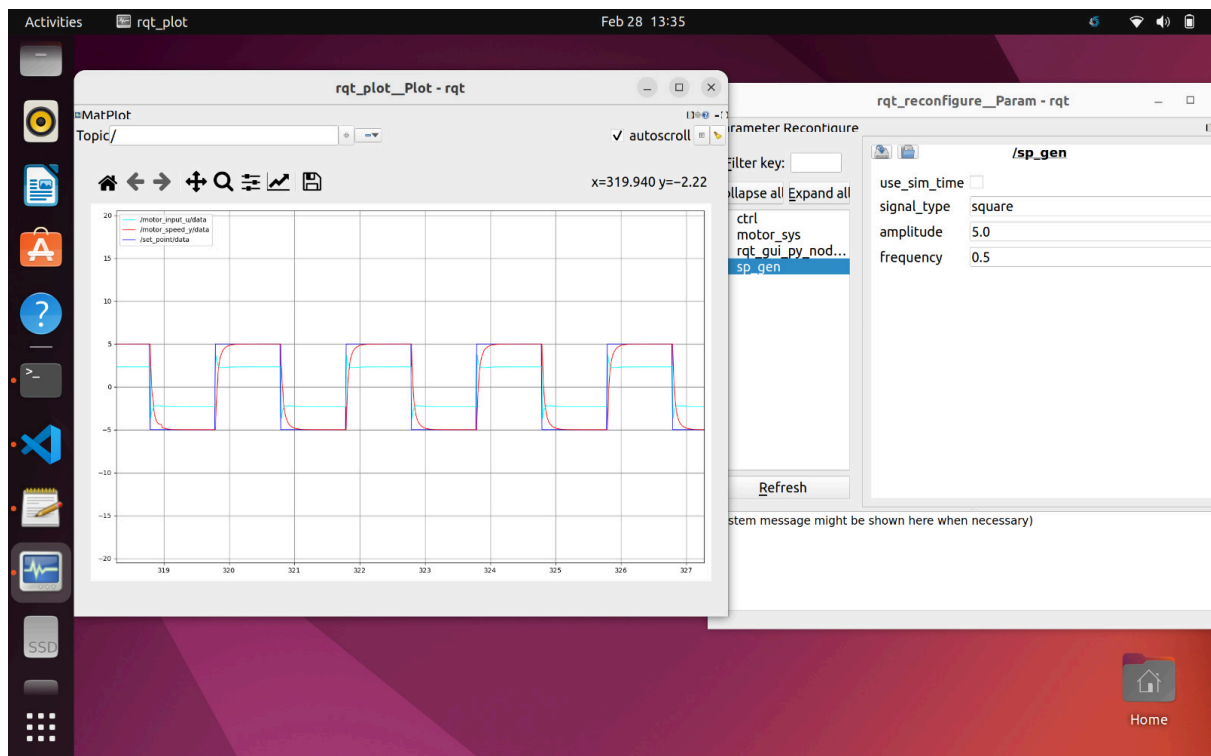


Como se observa en el diagrama que generamos (*rqt_graph*), el flujo de información es bidireccional y síncrono; el nodo */ctrl* procesa la diferencia entre la referencia de */set_point* y la velocidad real del motor para enviar una señal de mando al nodo *n_motor_sys*. Esta robustez estructural se complementa con la flexibilidad operativa visualizada en *rqt_reconfigure*, donde el uso de parámetros dinámicos permitió establecer ganancias de $K_p = 0.5$ y $K_i = 1$ en tiempo real, optimizando el comportamiento del sistema sin interrumpir la simulación.

De estos parámetros que elegimos se refleja directamente en la respuesta temporal obtenida en *rqt_plot*, donde se aprecia un seguimiento de trayectoria. La señal de velocidad del motor (línea roja) se superpone con precisión a la señal de referencia (línea azul), lo que valida que la acción integral implementada logró eliminar el error en estado estacionario. Asimismo, la estabilidad de la señal de control (línea cian) confirma que la lógica de Anti-windup y la sintonización de parámetros fueron las adecuadas para la dinámica de la planta ($K=2.16$, $\tau=0.05$), garantizando un sistema autónomo, estable y capaz de reaccionar eficazmente a cambios continuos en la velocidad deseada.



Para evaluar de manera exhaustiva el comportamiento y la robustez del sistema, se amplió la arquitectura del nodo generador de referencias (*sp_gen*). En lugar de utilizar variables estáticas, se implementó un sistema de parámetros dinámicos mediante la función *add_on_set_parameters_callback*. Esta configuración permite configurar en tiempo real las características de la señal (amplitud, frecuencia y morfología) a través de herramientas como *rqt_reconfigure*, eliminando la necesidad de detener la ejecución o recompilar el espacio de trabajo ante cada prueba.



Conclusiones

Tras la ejecución y validación del sistema de control para el motor de CD, se concluye que los objetivos planteados al inicio de este reto se cumplieron satisfactoriamente. El éxito del reto está en la correcta integración de la arquitectura de software con la teoría de control clásica; se logró diseñar un nodo controlador funcional en Python que, mediante el uso de NumPy, permite un procesamiento de señales eficiente y síncrono. La implementación del algoritmo PI fue fundamental para alcanzar la precisión requerida, ya que la acción integral permitió eliminar el error en estado estacionario que presentaba la planta ante cambios en la referencia.

El cumplimiento total de los objetivos se debe, en gran medida, al aprovechamiento de las herramientas avanzadas de ROS 2. El uso de parámetros dinámicos y la interfaz de **rqt_reconfigure** fueron piezas clave que permitieron la sintonización fina de las ganancias K_p y K_i en tiempo real, facilitando la búsqueda de un equilibrio entre rapidez de respuesta y estabilidad sin necesidad de interrumpir el flujo de la simulación. Asimismo, la incorporación de una técnica de Anti-windup mediante **np.clip** resultó esencial para mantener la integridad del sistema, evitando que la saturación del actuador provocará oscilaciones divergentes o comportamientos erráticos.

A pesar de los resultados positivos, una posible mejora a la metodología implementada sería la transición hacia un controlador PID completo. La acción derivativa permitiría anticipar la tendencia del error, lo que reduciría aún más el sobretiro (overshoot) en cambios bruscos de trayectoria y mejoraría el tiempo de asentamiento del motor. Además, para futuras iteraciones del proyecto.

Referencias Bibliográficas (APA)

- **Manchester Robotics.** (2024). *DC Motor Control Challenge 2: ROS 2 Humble Integration Guide*.
- **Open Robotics.** (2026). *ROS 2 Humble Hawksbill Documentation*. Recuperado de <https://docs.ros.org/en/humble/>
- **Ogata, K.** (2010). *Ingeniería de Control Moderna* (5ª ed.). Pearson Educación.
- **Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E.** (2020). *Array programming with NumPy*. *Nature*, 585(7825), 357-362.
- **Python Software Foundation.** (2026). *Python Language Reference, version 3.10*. Recuperado de <https://www.python.org/>